Public API Reference

System overview

PetPals provides a unified public catalog for browsing available pets across all shelter branches in the multi-shelter network. The public API serves five endpoints that enable adopters to discover, filter, and view detailed information about adoptable animals.

Key Features

- Multi-value filters: species, breed, size, age range, shelter
- Cascading dropdowns: breed dropdown filters based on selected species
- Server-side pagination: efficient data loading
- Comprehensive pet details: personality, compatibility, health info
- Public access: all endpoints require no authentication

Route → Controller → Service → Database

1. Backend API Endpoints

Base path: /api/v1 – All endpoints are public (no authentication required)

1.1 GET /api/v1/pets - List available pets with filtering and pagination

Returns a paginated catalog suitable for card-based UI with optional multi-value filtering by species, breed, size, age range, and shelter.

Query Parameters

- species (integer, repeatable, optional) - filters by speciesID, matching the /breeds endpoint's convention
- breed (string, repeatable, optional)
- size (string, repeatable, optional) - Small | Medium | Large
- minAge (integer, optional) - minimum age in MONTHS (inclusive) - pet must be at least these many months old
- maxAge (integer, optional) - maximum age in MONTHS (inclusive) - pet must be at most these many months old
- shelterID (integer, repeatable, optional)
- page (integer, optional, default 1)
- limit (integer, optional, default 20, max 100)

Response


```
{
  "success": true,
  "message": "Pets retrieved successfully",
  "data": [
    {
      "petID": 1,
      "petName": "Buddy",
      "petAge": "2 yr, 2 mo",
      "petPhoto": "https://...",
      "breed": { "breedName": "Labrador", "speciesName": "Dog" }
    }
```

```
    ],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 45,
      "totalPages": 3
    }

  }
```

Design Notes

- **Note:** petAge is a formatted string (e.g. "2 yr, 2 mo") computed server-side from the pet's date of birth at request time - it is not a stored database field, so it will always reflect the pet's current age.
- Always filters by adoptionStatus: "available" (returns only adoptable pets)
- Multiple values within a filter are OR'd together (e.g., species=1&species=2 = Dog OR Cat)
- Multiple filters are AND'd together (e.g., species + breed = species AND breed)
- Empty filter arrays are skipped - no WHERE clause added

1.2 GET /api/v1/pets/:id - Get detailed pet information

Returns comprehensive pet detail including personality, compatibility flags, medical notes, and shelter info. Returns 404 if pet not found.

Response

```
{
  "success": true,
  "message": "Pet retrieved successfully",
  "data": {
    "petID": 5,
    "petName": "Rocky",
    "petAge": "2 yr, 2 mo",
    "petSex": "M",
    "petColor": "Brown",
    "petHeight": 22.5,
    "petWeight": 70,
    "petDesc": "Friendly and energetic",
    "breed": { "breedID": 3, "breedName": "Bulldog", "speciesName": "Dog" },
    "shelter": { "shelterID": 1, "shelterName": "PetPals Downtown", "shelterAddress":
"123 Main St" },
    "compatibleWithChildren": true,
    "compatibleWithPets": false,
    "specialNeeds": false
  }
}
```

1.3 GET /api/v1/species - Get all animal species

Populates the species filter dropdown. No pagination. Ordered by speciesName ascending.

Response

```
{
  "success": true,
  "message": "Species retrieved successfully",
  "data": [
    { "speciesID": 1, "speciesName": "Cat" },
    { "speciesID": 2, "speciesName": "Dog" },
    { "speciesID": 3, "speciesName": "Rabbit" }
```

```
    ]
  }
```

1.4 GET /api/v1/breeds - Get breeds for selected species

Cascading dropdown: returns breeds only for specified species IDs. If no speciesID is provided, returns an empty array. Ordered by speciesName then breedName.

Query Parameters

- **speciesID** (integer, repeatable, optional) - if omitted, returns empty array

> *Example: ?speciesID=1&speciesID=2*

Response

```
{
  "success": true,
  "message": "Breeds retrieved successfully",
  "data": [
    { "breedID": 1, "breedName": "Bulldog", "speciesName": "Dog" },
    { "breedID": 2, "breedName": "German Shepherd", "speciesName": "Dog" },
    { "breedID": 3, "breedName": "Labrador", "speciesName": "Dog" }
  ]
}
```

1.5 GET /api/v1/shelters - Get all open shelters

Populates the shelter filter dropdown. No pagination. Ordered by shelterName ascending. Only returns shelters with status "Open".

Response

```
{
  "success": true,
  "message": "Shelters retrieved successfully",
  "data": [
    { "shelterID": 1, "shelterName": "PetPals Downtown" },
    { "shelterID": 2, "shelterName": "PetPals Brooklyn" }
  ]
}
```

1.6 GET /api/v1/shelters/nearby - List open shelters within a radius of a lat/lng point

Returns open shelters within a given radius of a lat/lng coordinate, ordered by distance ascending. Uses PostGIS ST_DWithin and ST_Distance. Distinct from GET /shelters (which lists all open shelters with no geographic filtering) - this endpoint scopes results to a specific location.

Query Parameters

- lat (number, required) - latitude, must be between -90 and 90
- lng (number, required) - longitude, must be between -180 and 180
- radius (number, optional, default 25) - search radius in kilometers

Response

```
{
  "success": true,
  "message": "Shelters retrieved successfully",
  "data": [
```

```
    {
      "shelterID": 2,
      "shelterName": "PetPals Brooklyn",
      "shelterAddress": "456 Bedford Ave",
      "shelterZIP": 11211,
      "distance": 2.14
    }
  ]
}
```

Design Notes

- Only returns shelters with status "Open" - same filter as GET /shelters
- distance is returned in kilometers, rounded to 2 decimal places, and computed via ST_Distance
- Results are ordered by distance ascending - nearest shelter first
- lat/lng outside valid range (lat outside -90 to 90, lng outside -180 to 180) returns 400 BAD_REQUEST
- This endpoint filters SHELTERS by location, not pets directly - to find pets near a location, call this endpoint first, then pass the returned shelterIDs into GET /pets

2. Key Design Patterns

2.1 Multi-Value Filtering

Query parameters can repeat to allow multiple filter values:

```
?species=Dog&species=Cat&breed=Bulldog&breed=Poodle
```

Express auto-converts repeated keys to arrays:

```
req.query.species = ["Dog", "Cat"]
req.query.breed = ["Bulldog", "Poodle"]
```

Semantics

- Multiple values within a filter are OR'd (species IN [Dog, Cat] = Dog OR Cat)
- Multiple filters are AND'd (species + breed = species AND breed)
- Empty filter arrays are skipped - no WHERE clause added

2.2 Cascading Dropdowns

Breed dropdown depends on species selection:

- Page load: GET /species → populate species dropdown
- User selects species: GET /breeds?speciesID=1&speciesID=2 → populate breed dropdown
- User selects breeds: GET /pets?species=Dog&breed=Bulldog → list results

> *Benefit: reduces irrelevant options; improves UX.*

2.3 Dynamic Prisma WHERE

A single dynamic where object is used instead of separate queries:

```
const where = { adoptionStatus: "available" };
const breedWhere = {};
```

```
if (breedValues.length > 0) breedWhere.breedName = matchFilter(breedValues);
if (speciesValues.length > 0) breedWhere.species = { speciesName:
matchFilter(speciesValues) };
if (Object.keys(breedWhere).length > 0) where.breed = breedWhere;
```

> *Benefits: no code duplication, easier to extend, clean merging.*

2.4 Age-Range Filtering via Date of Birth

Pets store petDOB (date of birth), not a static age, so age is always computed live rather than going stale. minAge/maxAge (in months) are converted into a petDOB date-range comparison before querying, via buildAgeFilter in pets.service.js:

```
const monthsAgo = (months) => {
  const today = new Date();
  return new Date(today.getFullYear(), today.getMonth() - months, today.getDate());
};

const buildAgeFilter = (minAge, maxAge) => {
  const dobFilter = {};
  if (minAge !== undefined) {
    dobFilter.lte = monthsAgo(Number(minAge));
  }
  if (maxAge !== undefined) {
    const today = new Date();
    const cutoff = new Date(
      today.getFullYear(),
      today.getMonth() - (Number(maxAge) + 1),
      today.getDate(),
    );
    dobFilter.gt = cutoff; // strict greater-than, not gte
  }
  return Object.keys(dobFilter).length > 0 ? dobFilter : undefined;
};
```

Why minAge is straightforward

"At least N months old" means the pet's DOB must fall on or before the date N months ago -- older pets have earlier birth dates, so the comparison is petDOB <= monthsAgo(N). This is a direct, inclusive (lte) comparison with no adjustment needed.

Why maxAge needs the +1 month and strict gt

A naive implementation of maxAge would mirror minAge exactly: dobFilter.gte = monthsAgo(maxAge). This looks correct but is wrong at the boundary. The bug was caught by testing maxAge=35 against a real pet (DOB 2023-07-10) whose displayed age correctly showed 35 months -- yet the naive filter excluded that pet.

The root cause: the function that computes a pet's displayed age (ageInMonthsFromDOB) rounds DOWN if today's day-of-month hasn't yet reached the DOB's day-of-month:

```
if (today.getDate() < dobDate.getDate()) months -= 1;
```

This means "35 months," as displayed, can correspond to a DOB almost a full month earlier than a naive "35 months back from today" calculation would assume. The naive maxAge cutoff and the displayed age were quietly using two different definitions of "35 months" and disagreed exactly at this kind of boundary.

The fix shifts the cutoff back one additional month AND uses a strict greater-than comparison instead of gte. Both changes are required together -- shifting the month back without the strict inequality (or vice versa) still produces mismatches. This exact combination was verified by sweeping every day-of-month against a wide range of ages, confirming zero mismatches against ageInMonthsFromDOB's own logic.

Important

> *The minAge and maxAge cutoffs are NOT symmetric. minAge uses (today - N months) directly with an inclusive lte comparison, but maxAge must shift back an additional month and use a strict gt comparison -- otherwise a pet exactly N months old is wrongly excluded at the maxAge boundary. This asymmetry comes from how whole-months-elapsed is calculated (a pet's age only increments once the calendar reaches its birth day-of-month each month).*
>
> *Note: this filtering logic is entirely separate from how petAge is formatted in the response. The filter operates only on dates (petDOB compared against calculated cutoff dates) and never touches the formatted petAge string -- the two are decoupled, so changing the response's display format has no effect on filtering, and vice versa.*

2.5 Pagination with Parallel Count

All list endpoints (currently just GET /pets) support page-based pagination via two query parameters:

- **page** (integer, optional, default 1) - which page of results to return
- **limit** (integer, optional, default 20, max 100) - how many results per page

Query Parameters vs. Response Fields - Query Parameters

- page (integer, optional, default 1, minimum 1)
- limit (integer, optional, default 20, minimum 1, maximum 100)

Validation

- Both page and limit must be whole integers - a non-integer value (e.g. page=abc, limit=2.5) returns 400 BAD_REQUEST
- page must be >= 1
- limit must be between 1 and 100 inclusive
- Important: out-of-range values are REJECTED, not clamped. A request with limit=500 does NOT silently return 100 results - it returns a 400 BAD_REQUEST error. The caller (frontend or otherwise) is responsible for staying within the valid range.

Defaults

- If page is omitted entirely, it defaults to 1
- If limit is omitted entirely, it defaults to 20
- These defaults live in the backend controller (pets.controller.js) - they are not sent by the frontend. The frontend's own request-side default (LIMIT = 20 in PetCatalog.tsx) happens to match, but the two are independent; changing one does not change the other.

Response Shape

Every paginated response includes a pagination object alongside data:

```
{
  "page": 1,
  "limit": 20,
  "total": 45,
  "totalPages": 3
}
```

- `total` is the total count of records matching the applied filters (not the total in the whole table)

- *totalPages* is calculated as `Math.ceil(total / limit`, used by the frontend to render Previous/Next controls and disable them at the boundaries.

Implementation: Parallel Count

The data fetch and count query run in parallel using Promise.all():

```
const [pets, total] = await Promise.all([
  prisma.pet.findMany({ where, skip, take: limit, select: {...} }),
  prisma.pet.count({ where })
]);
```

> *Allows the frontend to calculate totalPages for pagination controls.*

3. Backend Supporting Files

File / Layer	What happens here
server/src/services/pets.service.js	getAvailablePets(), getPetDetails(). Dynamic where building, pagination.
server/src/services/species.service.js	getSpecies(). All species ordered by name.
server/src/services/breeds.service.js	getBreeds(speciesIDs). Breeds for selected species.
server/src/services/shelters.service.js	getShelters(). All open shelters ordered by name.
server/src/controllers/pets.controller.js	getAvailablePets(req, res), getPetDetails(req, res).
server/src/controllers/species.controller.js **breeds.controller.js** **shelters.controller.js**	Validate query params and call the corresponding services.
server/src/routes/pets.routes.js	Registers all five endpoints. No authenticate middleware (public routes).
server/src/app.js	Mounts petsRouter at app.use("/api/v1", petsRouter).

4. Error Handling

Standard Error Response

```
{
  "success": false,
  "message": "Descriptive error message",
  "error": { "code": "ERROR_CODE", "details": "..." }
}
```

Common Error Codes

Scenario	HTTP Status	Response Message
BAD_REQUEST	400	Invalid query param type/value (non-integer page, size not in enum, minAge > maxAge)

Scenario	HTTP Status	Response Message
NOT_FOUND	404	Pet ID does not exist (GET /pets/:id)
INTERNAL_SERVER_ERROR	500	Unexpected DB or server error

5. Design Decisions & Rationale

- **Repeatable query parameters:**
  Standard REST practice; simpler than JSON body. Express auto-converts repeated keys to arrays.
- **AND/OR logic:**
  Aligns with user expectations ("Dogs AND Bulldogs" = Dogs that are Bulldogs). Multiple values within a filter naturally OR via the Prisma IN operator.
- **All endpoints public (no auth):**
  Catalog discovery is a primary entry point; unauthenticated browsing maximizes adoption potential.
- **Server-side pagination:**
  Efficient for large datasets; reduces payload size. Frontend caches pages independently via TanStack Query.
- **Separate dropdown endpoints:**
  Lightweight, reusable, and callable in parallel with pet listing.
- **Cascading breed dropdown:**
  Better UX; reduces cognitive load; enforces logical flow (select species first, then breed).
- **Location filtering deferred:**
  Reduces Sprint 2 scope. PostGIS proximity search planned for a future sprint

6. Quick Reference - All Endpoints

Method	Endpoint	Description
GET	/api/v1/pets	List available pets with filters + pagination
GET	/api/v1/pets/:id	Get detailed pet information
GET	/api/v1/species	Get all animal species
GET	/api/v1/breeds	Get breeds for selected species (cascading)
GET	/api/v1/shelters	Get all open shelters
GET	/api/v1/shelters/nearby	List open shelters within a radius of a lat/lng point

All endpoints above require no authentication.